Lesson 07 Demo 02

Implementing Error Handling in API Calls

Objective: To develop a React application and demonstrate error handling in API calls, showcasing best practices for managing failures, displaying fallback UI, and ensuring a smooth user experience

Tools Required: Visual Studio Code

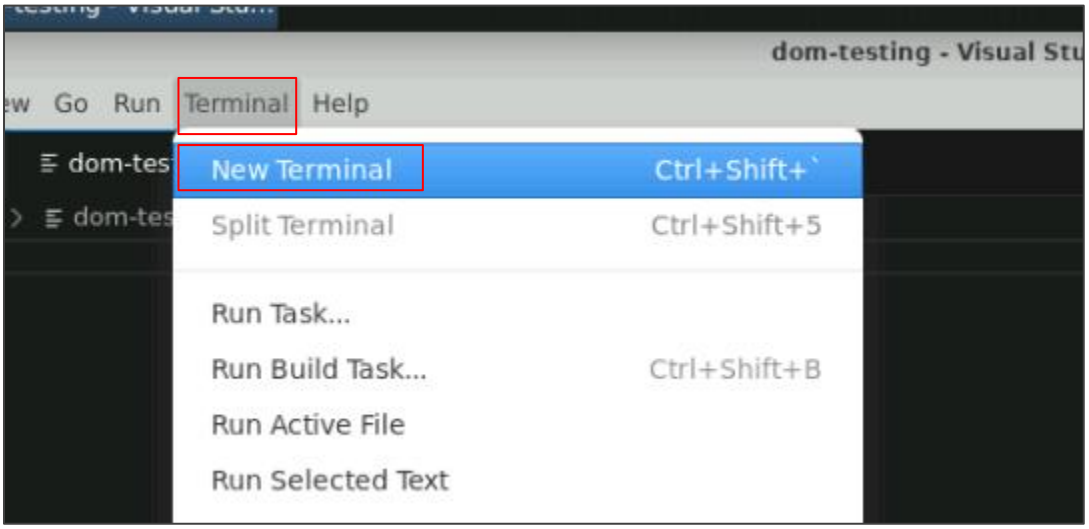
Prerequisites: NA

Steps to be followed:

1. Set up a new React project using Vite
2. Import and use the component
3. Run and verify the application

Step 1: Set up a new React project using Vite

1.1 Open Visual Studio Code, click on **Terminal,** and then select **New Terminal**



1.2 Open the terminal and run the following command:

npm create vite@latest error-demo

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

labuser@ip-172-31-12-132:~$ npm create vite@latest error-demo
```

1.3 Select **React** and press enter

```
> npx
> create-vite error-demo

◆ Select a framework:
  ○ Vanilla
  ○ Vue
  ● React
  ○ Preact
  ○ Lit
  ○ Svelte
  ○ Solid
  ○ Qwik
  ○ Angular
  ○ Others
```

1.4 Select **JavaScript** and press enter

```
◇ Select a framework:
  React

◆ Select a variant:
  ○ TypeScript
  ○ TypeScript + SWC
  ● JavaScript
  ○ JavaScript + SWC
  ○ React Router v7 ↗
```

1.5 Run the following command to navigate to the project folder:
cd error-demo/

```
labuser@ip-172-31-12-132:~$ cd error-demo/
labuser@ip-172-31-12-132:~/error-demo$
```

1.6 Run the following command to install dependencies:
npm i

```
labuser@ip-172-31-12-132:~/error-demo$ npm i
```

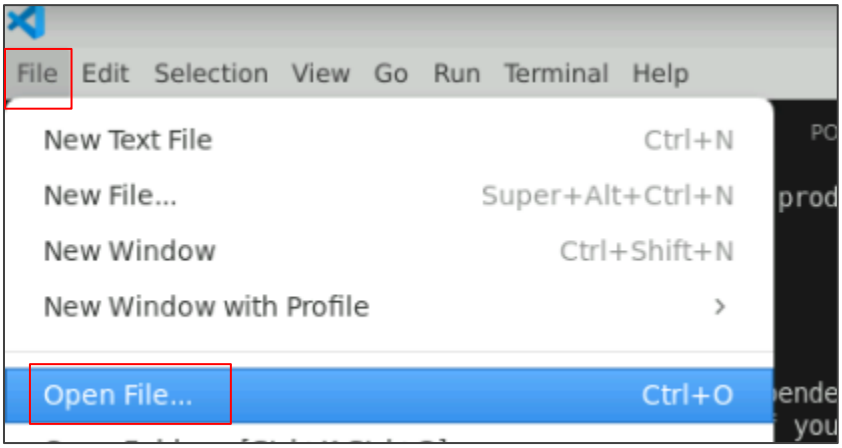
1.7 Run the following command to start the development server:
npm run dev

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS
labuser@ip-172-31-12-132:~/error-demo$ npm run dev
```

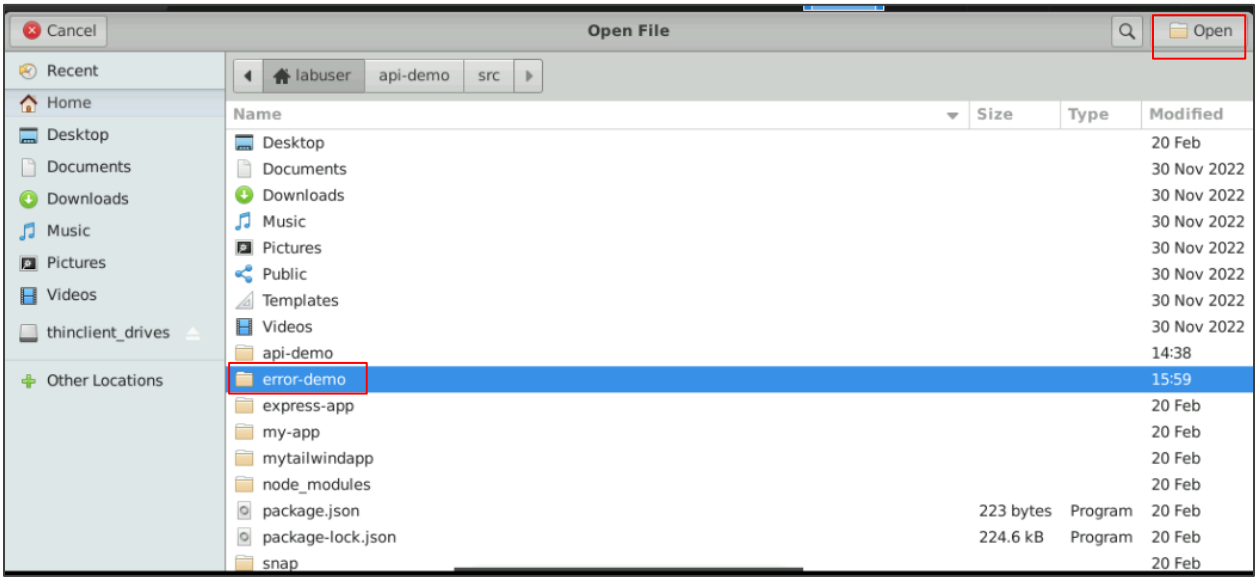
1.8 Navigate to the web browser and paste the link given below:
http://localhost:5173

Step 2: Import and use the component

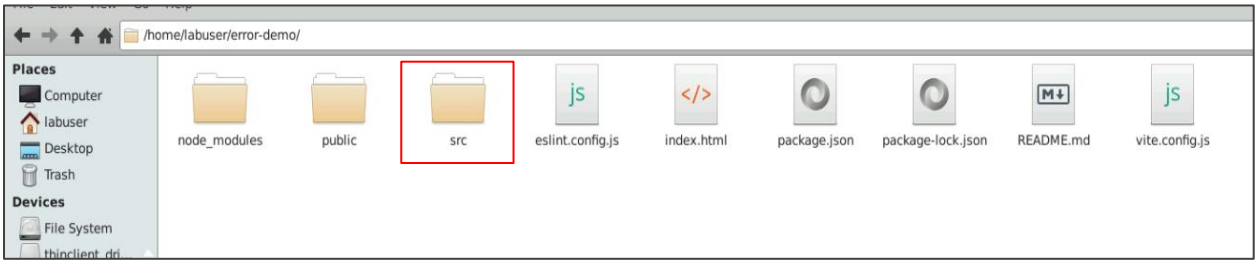
2.1 Open Visual Studio Code, click on **File**, and select **Open File…**



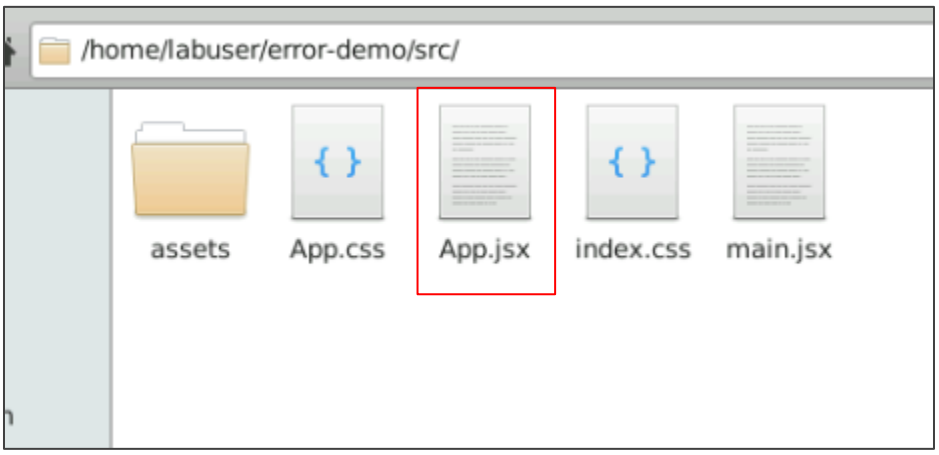
2.2 Select **error-demo** and click **Open**



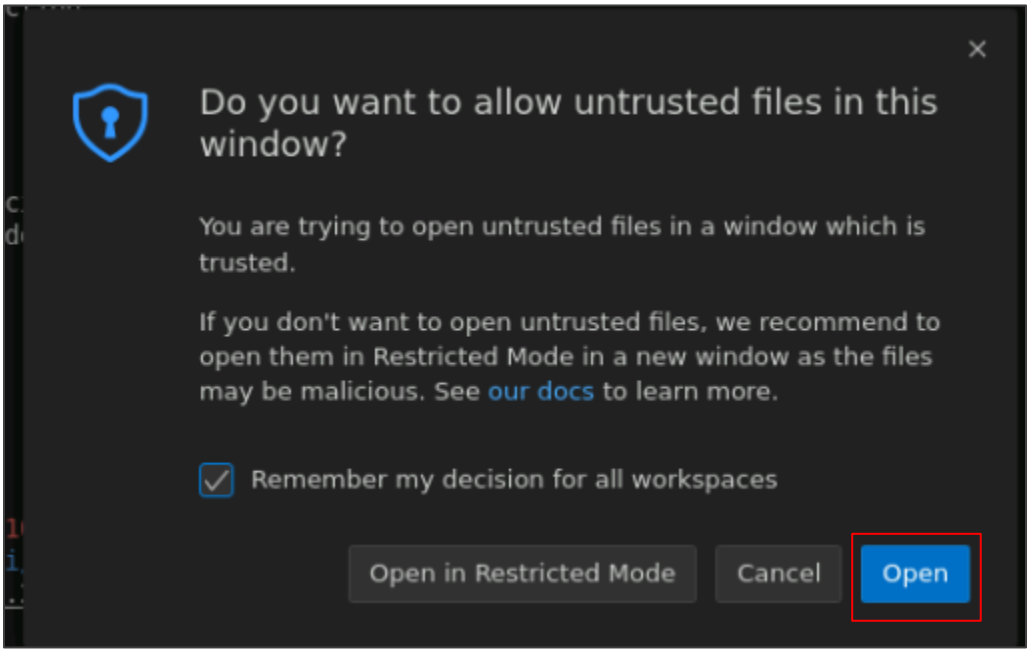
2.3 Double-click **src**



2.4 Open **App.jsx** by double-clicking on it



2.5 Click **Open**



2.6 Paste the code given below to include the new component in **App.jsx:**

```
import React, { useState, useEffect } from 'react';
import './App.css';

function App() {
  const [error, setError] = useState(null);
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);

  async function fetchData() {
    try {
      const response = await fetch('https://jsonplaceholder.typicode.com/todos/1');
      if (!response.ok) {
        throw new Error(`HTTP error! Status: ${response.status}`);
      }
      const jsonData = await response.json();
      setData(jsonData);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  }

  useEffect(() => {
    fetchData();
  }, []);

  return (
    <div className="App">
      <h1>Error Handling Demo</h1>
      {loading ? (
        <p>Loading...</p>
      ) : error ? (
        <p style={{ color: 'red' }}>Error: {error}</p>
      ) : (
        <pre>{JSON.stringify(data, null, 2)}</pre>
```

```
    )}
   </div>
  );
}

export default App;
```

```
ne > labuser > error-demo > src > App.jsx > ...
import React, { useState, useEffect } from 'react';
import './App.css';

function App() {
  const [error, setError] = useState(null);
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);

  async function fetchData() {
    try {
      const response = await fetch('https://jsonplaceholder.typicode.com/todos/1');
      if (!response.ok) {
        throw new Error(`HTTP error! Status: ${response.status}`);
      }
      const jsonData = await response.json();
      setData(jsonData);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
```

Step 3: Run and verify the application

3.1 Navigate to the browser and refresh the web page

By following these steps, you have successfully developed a React application using the Vite framework that demonstrates error handling in API calls. You applied best practices such as checking response status, managing errors using state, displaying fallback UI, and ensuring a seamless user experience.